

TALLER PRÁCTICO

Scrum en equipo: del Product Backlog a la Retrospectiva

Análisis de Sistemas · Proyecto RPG Manager

Jira o Keisen | Equipos de 3–4 | 2 horas | Semanas 11–12 |

Nombre del equipo	Módulo asignado	Fecha
ScrumTeam	Módulo 2 - Taller 1x	4/09/2026

Integrante	Rol Scrum	Usuario GitHub
1. Paula Camila Hurtado Ramírez	Product Owner	paulalalalal
2. Samuel Julián Castillo Muñoz	Scrum Máster	Samuel-uxn
3. Emily Sofia Castillo Preciado	Desarrollador	EmilyCastillo31
4. Santiago Abella Bello	Desarrollador	Santyabella
5. Juan Miguel Tapiero Gutiérrez	Desarrollador	Juan123-del

OBJETIVOS DEL TALLER

Al finalizar esta sesión el equipo será capaz de:

1. Distinguir un caso de uso UML de una historia de usuario y transformar el primero en el segundo.
2. Redactar historias de usuario con la plantilla estándar y criterios de aceptación verificables.
3. Priorizar un Product Backlog aplicando MoSCoW y justificar las decisiones ante el stakeholder.
4. Estimar esfuerzo relativo mediante Planning Poker usando la secuencia de Fibonacci.
5. Planear un Sprint respetando una capacidad fija y negociar el alcance cuando no todo cabe.
6. Ejecutar un ciclo Scrum completo: Sprint Planning, tablero Kanban, Sprint Review y Retrospectiva.
7. Configurar y operar un proyecto Scrum en la herramienta Keisen.

Distribución del tiempo

Bloque	Contenido	Tiempo
0	Formación de equipos, asignación de roles y módulo	10 min
1	Configuración de la herramienta (Jira o Keisen)	15 min

Bloque	Contenido	Tiempo
2	Redacción del Product Backlog: 8 historias de usuario	20 min
3	Priorización con MoSCoW	10 min
4	Estimación con Planning Poker	20 min
5	Sprint Planning con capacidad fija (13 puntos)	10 min
6	Simulación del Sprint en el tablero Kanban + incidentes	20 min
7	Sprint Review cruzada entre equipos	10 min
8	Retrospectiva y cierre	5 min
	TOTAL	120 min

Antes de empezar: caso de uso ≠ historia de usuario

En las semanas 5–6 documentaste casos de uso UML del RPG Manager. Hoy vas a trabajar con historias de usuario. No son sinónimos ni una es la versión "moderna" de la otra: sirven a momentos distintos del análisis y conviven en un mismo proyecto.

Aspecto	Caso de uso (UML)	Historia de usuario (Scrum)
Propósito	Describir con precisión CÓMO interactúa un actor con el sistema	Capturar QUÉ valor necesita alguien, para conversarlo después
Tamaño	Grande: cubre un objetivo completo del actor	Pequeña: debe caber en un Sprint
Estructura	Actor, precondiciones, flujo principal, flujos alternativos, postcondiciones	Una frase + criterios de aceptación
Cuándo se escribe	En la fase de análisis, antes de construir	Continuamente, durante todo el proyecto
Quién lo usa	Analista y arquitecto	Product Owner y equipo de desarrollo
Nivel de detalle	Alto desde el inicio	Bajo al inicio; se detalla al acercarse al Sprint

CONCEPTO CLAVE — La relación entre ambos

Un caso de uso normalmente se descompone en VARIAS historias de usuario. El caso de uso "Gestionar personaje" puede convertirse en: crear personaje, editar atributos, consultar ficha, eliminar personaje y validar nombre duplicado.

Esa descomposición es exactamente lo que harás en el Bloque 2. No estás empezando de cero: estás traduciendo trabajo de análisis que tu equipo ya hizo al lenguaje que el equipo de desarrollo puede planificar.

Módulos por equipo

Cada equipo trabaja un módulo distinto del RPG Manager. Al final de la sesión, entre todos los equipos habrá quedado documentado el backlog completo del sistema. Tu profesor te asignará uno:

Equipo	Módulo	Alcance funcional
E1	Gestión de personajes	Crear, editar, consultar y eliminar la ficha de un personaje
E2	Inventario y equipamiento	Añadir objetos, equipar armas y armaduras, controlar capacidad
E3	Arena de combate	Emparejar personajes, resolver turnos, registrar el resultado
E4	Misiones y progresión	Asignar misiones, otorgar experiencia, subir de nivel
E5	Ranking y estadísticas	Tabla de clasificación, historial de combates, métricas por clase
E6	Cuentas de jugador	Registro, inicio de sesión, perfil y recuperación de contraseña

Roles del equipo

Los roles se asignan al inicio y se mantienen durante toda la sesión. Escríbelos en la tabla de la primera página: forman parte de la evaluación.

Rol	Responsabilidad durante el taller	Entregable que produce
Product Owner	Redacta y ordena el backlog. Defiende la priorización ante el profesor, que actúa como stakeholder. Decide qué entra al Sprint.	Backlog priorizado con justificación
Scrum Master	Facilita las dinámicas, controla los tiempos de cada bloque, registra impedimentos y evita que una sola persona domine la conversación.	Registro de impedimentos + acta de retrospectiva
Developer(s)	Estiman en Planning Poker, descomponen historias en tareas y mueven las tarjetas en el tablero Kanban.	Estimaciones + tablero actualizado

REGLA DEL TALLER

El Product Owner NO estima. En Scrum quien hace el trabajo es quien estima el esfuerzo; si el PO estima, contamina la conversación con sus propias expectativas de plazo. Durante el Planning Poker el PO solo aclara dudas sobre el alcance de la historia.

BLOQUE 1 · Configuración de la herramienta

15 minutos

Este taller funciona con dos herramientas. Tu profesor indicará cuál usa el curso. Sigue únicamente la vía que te corresponde y luego continúa al Bloque 2: todo lo demás del taller es idéntico en ambas.

	Vía A · Jira Cloud	Vía B · Keisen
Configuración previa	Ninguna: el backlog está activo desde el inicio	Obligatoria: 5 pasos de Project Setup
Sprint formal	Sí (Start sprint / Complete sprint)	Sí
Planning Poker	No integrado: se hace con tarjetas físicas	Integrado en la herramienta
Gráfica de avance	Sí (burndown chart en Reports)	Métricas básicas
Límite de usuarios	10 por sitio en el plan gratuito	Sin límite relevante

Cada equipo trabaja en su propio sitio de Jira. El plan gratuito admite 10 usuarios por sitio, de modo que un equipo de 4 más el profesor caben holgadamente.

Paso	Acción	Valor a usar
1	Un integrante crea el sitio en `atlassian.com` con su correo	Guarda la URL: ` <nombre>.atlassian.net`< td=""></nombre>.atlassian.net`<>
2	Invita a los demás integrantes y al profesor	Settings → People → Add people
3	Crea el proyecto eligiendo plantilla	Software → `Scrum` Nombre: `RPG Manager – [tu módulo]`
4	Activa la estimación en puntos	Board settings → Estimation → `Story Points`
5	Crea el Sprint	En Backlog: `Create sprint` → nombre `Sprint 1`
6	Configura el límite WIP de la columna In Progress	Board settings → Columns → Max issues = `2`

⚡ TIP — Jira

Los pasos 4 y 6 son los únicos que hay que hacer antes de empezar; si se te olvidan, el Bloque 4 y el Bloque 6 no se pueden registrar en la herramienta.

El plan gratuito permite un solo sitio por cuenta. Si tu correo ya tiene un sitio de otra materia, usa el de otro integrante.

La sección "Quickstart" del panel derecho se puede cerrar con "Dismiss Quickstart": ocupa espacio y no la necesitas.

1.B · Vía Keisen

En Keisen el botón para crear la primera User Story aparece deshabilitado hasta que el proyecto esté configurado. La barra "Project Setup — 0 of 5 steps completed" es lo primero que hay que resolver.

Paso	Qué configurar	Valor a usar en este taller
1	Nombre y descripción del proyecto	`RPG Manager - [tu módulo]` Descripción: sistema de gestión de personajes de rol
2	Framework de trabajo	`Scrum` (no elijas Kanban puro: necesitamos Sprints)
3	Equipo y miembros	Los 3–4 integrantes, con el nombre real de cada uno
4	Roles Scrum	Product Owner, Scrum Master y Developers según la tabla anterior
5	Configuración del Sprint	Nombre: `Sprint 1` Duración: la más corta disponible Capacidad: 13 story points

⚡ TIP — Si el botón sigue deshabilitado

Verifica que la barra marque "5 of 5 steps completed". Si algún paso quedó a medias, Keisen lo mantiene bloqueado sin avisar cuál falta. Despliega la barra y revisa paso por paso.

Si al terminar el setup el ícono de la hoja con + sigue gris, recarga la página: la interfaz no siempre refresca el estado del botón por sí sola.

Sesión abierta en un solo navegador por equipo. Si dos integrantes editan a la vez, los cambios de uno pueden sobrescribir los del otro.

📋 PLAN B — Si la herramienta falla

Si la aplicación no carga, no responde o la configuración se queda bloqueada, el taller continúa igual en papel. No pierdas tiempo esperando: avisa al profesor y pasa al plan analógico.

Necesitas: notas adhesivas de tres colores (una por columna del Kanban), un marcador y un pliego de papel o una sección del tablero.

Dibuja tres columnas: To Do · In Progress · Done. Cada historia es una nota adhesiva con su código (US01), su título y sus story points en la esquina.

Todo lo demás del taller (redacción, MoSCoW, Planning Poker, review, retrospectiva) funciona idéntico. La herramienta acelera el registro, pero no es el contenido de la sesión.

BLOQUE 2 · Redacción del Product Backlog

20 minutos

El Product Owner escribe, pero todo el equipo aporta. La meta son 8 historias del módulo asignado.

2.1 La plantilla

```
Como <tipo de usuario>,  
quiero <realizar una acción>,  
para <obtener un beneficio>.
```

CONCEPTO CLAVE — La tercera línea es la importante

El "para..." es lo que distingue una historia de usuario de una tarea técnica disfrazada. Si no logras escribir un beneficio real, probablemente estás describiendo una decisión de implementación, no una necesidad de alguien.

Compara: "Como usuario quiero un formulario con seis campos" no dice para qué sirve. "Como jugador quiero registrar mi personaje para empezar a jugar sin perder mi progreso" sí lo dice, y deja libertad al equipo sobre cómo construirlo.

2.2 Criterios de aceptación

Cada historia necesita al menos dos criterios de aceptación: condiciones verificables que permiten decidir sin discusión si la historia está terminada o no. Se escriben en formato Dado / Cuando / Entonces.

US01 — Crear personaje

```
Como jugador,  
quiero crear un personaje eligiendo su clase,  
para comenzar una partida con las habilidades que prefiero.
```

Criterios de aceptación:

CA1: Dado que ingreso un nombre no vacío y elijo una clase válida,
cuando confirmo la creación,
entonces el personaje aparece en mi lista con nivel 1.

CA2: Dado que ya tengo un personaje con el nombre "Ragnar",
cuando intento crear otro con el mismo nombre,
entonces el sistema muestra un error y no lo crea.

2.3 Criterio INVEST

Antes de dar por buena una historia, verifícala contra estas seis condiciones:

Letra	Significa	Pregunta de control
I	Independiente	¿Se puede construir sin esperar a otra historia?
N	Negociable	¿Deja abierto el cómo, o ya impone la solución técnica?
V	Valiosa	¿Alguien nota la diferencia cuando esté lista?
E	Estimable	¿El equipo entiende lo suficiente para ponerle puntos?
S	Pequeña (Small)	¿Cabe holgadamente en un Sprint?
T	Verificable (Testable)	¿Sus criterios de aceptación se pueden comprobar objetivamente?

EJERCICIO 2A · Redacta el backlog de tu módulo

Escribe 8 historias de usuario de tu módulo asignado. Complétalas primero en esta hoja y luego regístralas en Keisen (o en las notas adhesivas si aplicaste el Plan B). Deja las columnas de MoSCoW y puntos vacías por ahora: son de los bloques 3 y 4.

ID	Título	Como... quiero... para...	MoSCoW	Pts
US01	Objeto Faltante	Como jugador, quiero ver la lista de objetos de mi inventario, para saber qué tengo disponible durante la aventura.	Must	3
US02	Buscar objeto	Como jugador quiero tener un sistema de filtrado en el inventario, para que se me sea más fácil buscar algo.	Must	8
US03	Estadísticas	Como jugador quiero poder comparar las estadísticas de un equipo nuevo con el equipo actual que tengo, para poder ver la mejora.	Should	3
US04	Inventario lleno	Como jugador, quiero que el sistema me avise cuando mi inventario está lleno, para decidir qué descartar antes de seguir recogiendo objetos.	Should	5
US05	Objeto protegido	Como jugador quiero marcar un objeto como protegido, para no perderlo si fallo una misión.	Should	3
US06	Objetos obtenidos	Como jugador quiero tener un sistema de inventario, para conocer que objetos he recogido en la aventura.	Could	2
US07	Equipamiento de armas y armaduras	Como jugador, quiero equipar armas y armaduras de mi inventario, para mejorar las estadísticas de mi personaje.	Must	3
US08	Almacenamiento de inventario	Como jugador, quiero conocer la capacidad disponible de mi inventario, para saber si puedo almacenar nuevos objetos.	Wont	2

HISTORIAS A CORREGIR

Las tres historias siguientes están mal escritas. Identifica qué regla incumple cada una y reescríbela correctamente.

A) "Como desarrollador, quiero usar una tabla MySQL indexada por id_personaje para mejorar el rendimiento."

Regla incumplida: Negociable

Reescritura: Como jugador, quiero que mi ficha de personaje cargue rapido, para no esperar al entrar a jugar.

B) "Como jugador, quiero gestionar todo el sistema de personajes, inventario, combate y ranking."

Regla incumplida: Small

Reescritura: Como jugador, quiero equipar un arma de mi inventario, para aumentar mi poder de ataque en combate.

C) "Como jugador, quiero que la aplicación sea rápida e intuitiva."

Regla incumplida: Verificable

Reescritura: Como jugador, quiero que la pantalla de inventario cargue en menos de 2 segundos, para no perder el ritmo de juego.

BLOQUE 3 · Priorización con MoSCoW

10 minutos

El Product Owner propone una clasificación para cada historia y el equipo la discute. La regla que hace útil este ejercicio: no todas pueden ser Must have.

Categoría	Qué significa	Cuota máxima
Must have	Sin esto el módulo no sirve para nada. Es el mínimo que hace funcionar el sistema.	ori Máximo 3 hist as
Should have	Importante y esperado, pero el módulo funciona sin ello por un tiempo.	Máximo 3 historias
Could have	Mejora la experiencia. Se hace si sobra capacidad.	Libre
Won't have	Reconocido como deseable, pero explícitamente fuera de este alcance.	Al menos 1 historia

CÓMO SE REGISTRA EN LA HERRAMIENTA

En Jira: usa el campo Labels de cada historia con los valores Must, Should, Could o Wont. No uses el campo Priority: su escala (Highest...Lowest) tiene otra semántica y se presta a confusión.

En Keisen: usa el campo Priority de la historia según la equivalencia Must → High, Should → Medium, Could → Low.

POR QUÉ HAY CUOTAS

En proyectos reales, la falla más común de MoSCoW es que todo termina marcado como Must have. Cuando todo es prioritario, nada lo es, y la priorización deja de aportar información.

Obligarte a dejar al menos una historia en Won't have te fuerza a tomar una decisión explícita y defendible, que es justamente la habilidad que se evalúa hoy.

EJERCICIO 3A · Defensa ante el stakeholder

El profesor pasará por cada equipo actuando como stakeholder y preguntará al Product Owner: "¿Por qué esta historia es Must have y esta otra no?". Prepara la respuesta escribiéndola aquí.

Historia Must have más discutible: USO7: Equipamiento de armas y armadura

Justificación: No se tenía claro si las armaduras solo eran para guardar en el inventario o si el usuario podía equipar esa armadura a su personaje.

Historia en Won't have y razón de su exclusión: USO8: Almacenamiento de inventario

La estructura de ambas historias tenían funciones similares y se descartó al observarse que era redundante a la historia 4.

BLOQUE 4 · Estimación con Planning Poker

20 minutos

Los story points no miden horas: miden esfuerzo relativo, considerando complejidad, volumen de trabajo e incertidumbre. Una historia de 4 puntos es aproximadamente el doble de trabajo que una de 2, sin importar cuántas horas tome a cada persona.

4.1 La escala.

1 2 3 5 8 13 ?

1 = trivial, casi mecanico

2 = sencillo, ya sabemos como hacerlo

3 = trabajo real, sin sorpresas esperadas

5 = complejo o con partes que no dominamos

8 = muy grande: conviene partirla en dos historias

13 = demasiado grande: hay que dividirla obligatoriamente

? = no entiendo la historia, necesito que el PO la explique

POR QUÉ FIBONACCI Y NO 1,2,3,4,5

Los saltos crecen porque la precisión de una estimación cae a medida que el trabajo crece. Discutir si algo vale 8 o 9 puntos es tiempo perdido: en ese rango la incertidumbre es mayor que la diferencia.

La escala te obliga a elegir entre valores claramente distintos, lo que hace la conversación más rápida y honesta.

4.2 Reglas de la ronda

Paso	Qué ocurre
1	El Product Owner lee la historia y sus criterios de aceptación en voz alta.
2	El equipo pregunta lo que necesite. El PO aclara alcance, no soluciones técnicas.
3	Cada Developer elige un valor en silencio, sin decirlo ni mostrarlo.
4	A la cuenta de tres, todos revelan su carta simultáneamente.
5	Si todos coinciden, ese es el valor. Se registra y se pasa a la siguiente historia.
6	Si hay diferencia, quien puso el valor más alto y quien puso el más bajo explican su razonamiento (máximo 1 minuto cada uno). Se repite la votación.
7	Si tras dos rondas no hay acuerdo, se toma el valor más alto y el Scrum Master lo anota como riesgo.

LA REGLA DEL SILENCIO

Nadie dice un número antes de revelar. El objetivo de la simultaneidad es evitar el sesgo de anclaje: si alguien con más experiencia dice "esto es un 3", el resto tenderá a converger a 3 aunque piense distinto, y se pierde precisamente la información que hace valiosa la discusión.

El Scrum Master es responsable de hacer cumplir esta regla.

EJERCICIO 4A · Registra las estimaciones

Vuelve a la tabla del Ejercicio 2A y completa la columna Pts. Luego registra aquí las dos historias donde hubo más desacuerdo: son las más informativas del ejercicio.

Historia	Valor más bajo	Valor más alto	Valor final	¿Qué explicaba la diferencia?
USO-2	5	8	8	Al inicio los 2 developers coincidieron que era difícil ya que los objetos había que filtrarlos. Pero el developer que dio más

Historia	Valor más bajo	Valor más alto	Valor final	¿Qué explicaba la diferencia?
				valor explicó que la dificultad subía porque el filtro era por categoría lo cual dificultaba todo, y llegaron a la conclusión de darle calificación de 8.
USO-4	3	5	5	Acá el que developer que dio valor mas bajo dijo que era facil ver si el inventario estaba lleno ya que era recorrer una lista, pero los otros developers dijeron que aparte de ver si el inventario estaba lleno tocaba dar alertas para no permitir más objetos en el inventario

BLOQUE 5 · Sprint Planning con capacidad fija

10 minutos

La capacidad del equipo para este Sprint es de 13 story points. Ni uno más. Esta restricción no es arbitraria: reproduce la situación real en la que el tiempo del equipo es finito y el backlog siempre es más grande que el Sprint.

5.1 Cómo se llena el Sprint

El Product Owner propone historias en orden de prioridad. Los Developers suman puntos. Cuando la siguiente historia ya no cabe, el Sprint se cierra: no se admite "la metemos y vemos si alcanza".

Ejemplo de un Sprint que cierra en 13:

```
US01  Crear personaje ..... Must    5  ->  total   5
US03  Consultar ficha ..... Must    3  ->  total   8
US02  Editar atributos ..... Must    3  ->  total  11
US05  Validar nombre duplicado ... Should 2  ->  total  13   [LLENO]

US04  Eliminar personaje ..... Should 5  ->  NO CABE
US06  Historial de cambios ..... Could  8  ->  NO CABE
```

EL MOMENTO IMPORTANTE DEL TALLER

Cuando una historia Must have no cabe en el Sprint, tienes tres salidas y ninguna es gratis: dividir la historia en una parte mínima que sí quepa, cambiar su prioridad y aceptar que espere al siguiente Sprint, o sacar otra historia ya comprometida para hacerle sitio.

Lo que un equipo Scrum nunca hace es aumentar la capacidad para que todo entre. Esa decisión es la que el profesor evaluará en la Review.

EJERCICIO 5A · Tu Sprint Backlog

ID	Historia	MoSCoW	Puntos	Acumulado
US2	Buscar Objeto: Como jugador quiero tener un sistema de filtrado en el inventario, para que se me sea más fácil buscar algo.	Must	8	8
US7	Como jugador, quiero equipar armas y armaduras de mi inventario, para mejorar las estadísticas de mi personaje.	Must	3	11

ID	Historia	MoSCoW	Puntos	Acumulado
US6	Como jugador quiero tener un sistema de inventario, para conocer que objetos he recogido en la aventura.	Could	2	13
TOTAL DEL SPRINT				13/ 13

Historias que quedaron fuera y por qué:

US01 (Objeto Faltante) quedó fuera a pesar de ser Must have, porque US02 (Buscar objeto), también Must, consumió 8 puntos, dejando espacio solo para US07. Esto es un caso real de que dos historias Must have no cupieron juntas en la capacidad del Sprint; para la siguiente sesión, el equipo debería revisar si US02 puede dividirse en algo más pequeño, ya que un puntaje de 8 sugiere que probablemente incluye más de una funcionalidad.

US08 no compitió por espacio en el Sprint, ya que fue clasificada como Won't have desde el MoSCoW, reconocida como deseable pero fuera de este alcance.

US01 y US08 tienen contenido relacionado (ambas tocan la capacidad/disponibilidad del inventario), pero se mantuvieron como historias separadas por su distinta prioridad: US01 es esencial para el módulo Must, mientras que US08 es una mejora reconocida pero no crítica Won't.

Además al priorizar las 2 historias must, US02 y US08, solo entraba en el rango de 13 una historia de 2 puntos, habien 2 de dos punto y una era wont, y entonces esa se descartó.

BLOQUE 6 · Simulación del Sprint en el tablero Kanban

20 minutos

El Sprint se simula por rondas. En cada ronda cada Developer mueve una tarjeta y el Scrum Master anota lo ocurrido. No se escribe código: se practica el flujo de trabajo y la toma de decisiones cuando algo sale mal.

Columna	Significado	Límite WIP
To Do	Comprometida en el Sprint, sin empezar	Sin límite
In Progress	Alguien está trabajando en ella ahora mismo	Máximo 2 tarjetas
Done	Cumple TODOS sus criterios de aceptación	Sin límite

⚠ LÍMITE WIP: MÁXIMO 2 EN PROGRESO

Aunque el equipo tenga 4 personas, solo puede haber 2 historias en curso a la vez. Si quieres empezar una tercera, primero tienes que terminar alguna de las que ya están abiertas.

Esta restricción existe porque el trabajo a medias no entrega valor: veinte historias al 80% valen exactamente cero al final del Sprint. El límite WIP obliga al equipo a terminar antes de empezar.

En Jira el límite se configura en Board settings → Columns → Max issues = 2, y la columna se marca en rojo al superarlo. Si usas papel, el Scrum Master hace cumplir la regla.

6.1 Incidentes del Sprint

A mitad del Sprint el profesor entregará a cada equipo dos de los siguientes incidentes. El equipo tiene 3 minutos para decidir qué hace y el Scrum Master registra la decisión.

#	Incidente	Lo que se evalúa
1	Un integrante falta a la sesión de hoy. La capacidad del Sprint baja a 9 puntos.	¿Qué historia se saca y quién decide sacarla?
2	El stakeholder pide con urgencia una historia nueva de 5 puntos que no estaba en el backlog.	¿Se acepta un cambio de alcance a mitad de Sprint?
3	Una historia estimada en 3 resultó ser de 8 al empezar a trabajarla.	¿Se reestima, se divide o se devuelve al backlog?
4	Dos historias que creían independientes resultaron acopladas: una no funciona sin la otra.	¿Cómo afecta esto al criterio INVEST y al orden del tablero?
5	Una historia llegó a Done pero no cumple uno de sus criterios de aceptación.	¿Vuelve a In Progress? ¿Cuenta como completada en la Review?



EJERCICIO 6A · Registro de incidentes (Scrum Master)

Incidente recibido n.º 1 Falta un integrante, capacidad baja de 13 a 9 **Decisión del equipo:**

- El equipo decidió sacar la historia **USO2**, de 8 puntos ya que al faltar un integrante coincidieron que no iba a lograr acabar esa historia al final del sprint al ser compleja de desarrollar.

-La segunda opción fue que solo habian 3 historias que sumaban 13->(8+3+2) y el limite del sprint solo admite 9 punto, si no se descartaba la historia de 8 puntos su suma total exedia los 9 puntos totales.

Incidente recibido n.º 3 Historia que creció de 3 a 8 puntos **Decisión del equipo:**

- Se decidió dividir la historia para no forzar la capacidad del Sprint. Si al dividirla se seguía excediendo el límite de 13 puntos, se devolvía al backlog la historia con menor prioridad para hacerle espacio.

BLOQUE 7 · Sprint Review cruzada

10 minutos · 5 minutos por cada dirección

Los equipos se emparejan (E1↔E2, E3↔E4, E5↔E6). Cada equipo presenta su Sprint al otro, que actúa como stakeholder y hace preguntas. Luego se invierten los papeles.

EN JIRA: EL CIERRE DEL SPRINT ES LA EVIDENCIA

Antes de presentar, el Scrum Master pulsa Complete sprint. Jira mostrará cuántas historias quedaron sin terminar y preguntará a dónde moverlas: al backlog o a un nuevo sprint.

Esa pantalla es la mejor prueba de si el equipo estimó bien. Preséntala junto con el burndown chart (menú Reports → Burndown Chart) durante la Review.

7.1 Qué presenta el equipo (3 minutos)

Momento	Contenido
1	Módulo trabajado y objetivo del Sprint en una sola frase.
2	Historias que llegaron a Done, con sus criterios de aceptación cumplidos.
3	Historias que quedaron fuera del Sprint y la razón de la exclusión.
4	Incidentes recibidos y qué decidió el equipo ante cada uno.

7.2 Qué pregunta el equipo evaluador (2 minutos)

Como stakeholder, debes hacer al menos dos de estas preguntas y anotar la respuesta:

Pregunta	Respuesta del equipo evaluado
¿Por qué esa historia es Must have y no Should have?	Porque sin equipar armas y armaduras el módulo no sirve de nada, es literalmente la razón de ser del inventario. Sin eso el jugador solo tendría objetos guardados sin poder usarlos.
¿Cómo saben que esa historia está realmente terminada?	Porque cumple sus dos criterios de aceptación, por ejemplo que el objeto aparezca en la ranura equipada y que si ya había uno puesto, se devuelva solo al inventario.
Si tuvieran 3 puntos más de capacidad, ¿qué historia entraría y por qué esa?	Entraría US01 de objeto Faltante, porque era Must have y se quedó por fuera solo porque US02 se comió casi toda la capacidad con 8 puntos. Con 3 puntos más ya hubiera cabido sin sacar nada.
¿Qué historia les resultó más difícil de estimar y qué la hacía incierta?	US02 (Buscar objeto), porque al principio parecía simple pero después vimos que tenía más partes de las que pensábamos como hacer varios filtros, por eso

Pregunta	Respuesta del equipo evaluado
	terminó en 8 puntos y no en algo más bajo como esperábamos.

BLOQUE 8 · Retrospectiva

5 minutos

La retrospectiva no evalúa el producto: evalúa la forma de trabajar del equipo. Cada integrante aporta al menos una idea. El Scrum Master facilita y escribe.

START — qué empezaríamos a hacer	STOP — qué dejaríamos de hacer	CONTINUE — qué mantendríamos
Master(Samuel): Revisar que cada historia tenga sus criterios de aceptación completos antes de pasar al Planning Poker, para no estimar algo que todavía no está bien definido.	Dejar que se acumulen varias correcciones pendientes en el backlog sin resolverlas antes de avanzar a los siguientes bloques.	Explicar el flujo completo antes de que el equipo empiece a trabajar, para que todos lleguen con el mismo nivel de claridad.
P.Owner(Paula): Dejar claro los backlogs ante cualquier duda de especificaciones en los CA, dejar registro de las puntuaciones y Moscow proporcionadas por el equipo.	Dejar de asumir que todos los integrantes tienen la misma interpretación de una historia de usuario o de sus criterios de aceptación. Evitar también modificar o aclarar requisitos sin dejar registro de los cambios acordados con el equipo.	Mantener la comunicación constante con el equipo para resolver dudas sobre los requisitos, priorizar el backlog según las necesidades del producto y tomar en cuenta las opiniones del equipo durante la planificación y estimación de las historias.
Developer(Emily): Apoyarme activamente en las estimaciones del Planning Poker revisando la viabilidad técnica y los criterios de aceptación desde el rol de desarrollo, asegurando que el equipo no sobrestime o subestime la complejidad de cada historia antes de llevarla al tablero.	Dejar de asumir que una historia de alto puntaje debe abordarse en un solo bloque sin evaluar previamente su división, evitando así que historias complejas bloqueen la capacidad del Sprint.	Mantener la colaboración continua y la discusión simultánea durante la votación de historias para alinear criterios técnicos sobre el código y la arquitectura con los demás desarrolladores.
Developer(Santiago): Debemos empezar a planificar mejor las tareas antes de iniciar el sprint definiendo que debía hacer cada integrante y empezaríamos a documentar los cambios realizados durante	Dejaríamos de realizar cambios sin consultar a los demás integrantes del equipo para evitar conflictos y creo que dejaríamos de asumir que una funcionalidad está terminada sin probarla correctamente.	Mantendríamos la comunicación constante entre los miembros del equipo para agilizar el trabajo y mantendríamos en regla la organización scrum para dividir las tareas de la mejor manera.

START — qué empezaríamos a hacer	STOP — qué dejaríamos de hacer	CONTINUE — qué mantendríamos
el desarrollo para facilitar el trabajo en equipo. Developer(Juan):Revisar los requerimientos y las dependencias técnicas antes de iniciar cada trabajo, para reducir los errores	Empezar el desarrollo antes de definir completamente que hace una historia	Mantener la comunicación entre desarrolladores, para compartir avances, resolver problemas y facilitar la integración

UNA ACCIÓN, NO UNA LISTA DE QUEJAS

Una retrospectiva que solo produce observaciones no cambia nada. Cierra eligiendo UNA sola acción concreta que el equipo se compromete a aplicar en la próxima sesión, con un responsable y algo verificable.

Ejemplo débil: "comunicarnos mejor". Ejemplo útil: "el Scrum Master avisará a los 10 minutos de cada bloque para que no se nos pase el tiempo redactando".

Acción comprometida para la próxima sesión:

Antes de cerrar el Planning Poker, el equipo revisará si alguna historia salió con 8 o más puntos, y evaluará dividirla en historias más pequeñas antes de pasar al Sprint Planning, para evitar que una sola historia grande deje fuera a otras de igual o mayor prioridad.

Responsable: Samuel Julián Castillo Muñoz (Scrum Master)

Entregable y registro

El taller se entrega al final de la sesión. No se recibe nada después de que el grupo salga del salón.

#	Entregable	Formato
1	Este documento completo, con todas las secciones diligenciadas	Físico, firmado por los integrantes
2	Captura del Product Backlog con las 8 historias estimadas	Imagen en el repositorio
3	Captura del tablero al cierre del Sprint (y burndown si usas Jira)	Imagen en el repositorio
4	Acta de retrospectiva con la acción comprometida	Archivo `retrospectiva.md`

REGISTRO EN GITHUB

Sube las capturas y el acta al repositorio del equipo antes de salir. Usa exactamente este formato de mensaje:

```
git commit -m "[semana-11] scrum-backlog-sprint-modulo-<tu-modulo>"
```

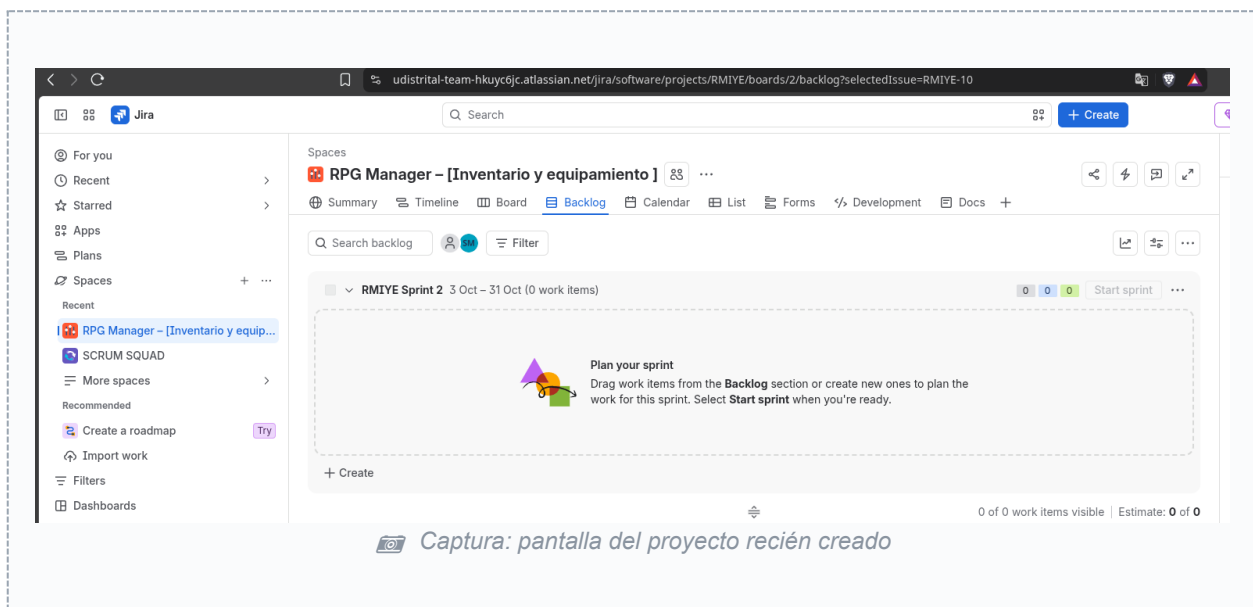
Sin commit visible antes de salir del salón, la sesión no cuenta para ninguno de los integrantes del equipo.

Anexo A · Configuración de Jira paso a paso

Solo para equipos que usan la Vía A. Cada paso indica la ruta exacta de menús. Marca la casilla cuando lo completes.

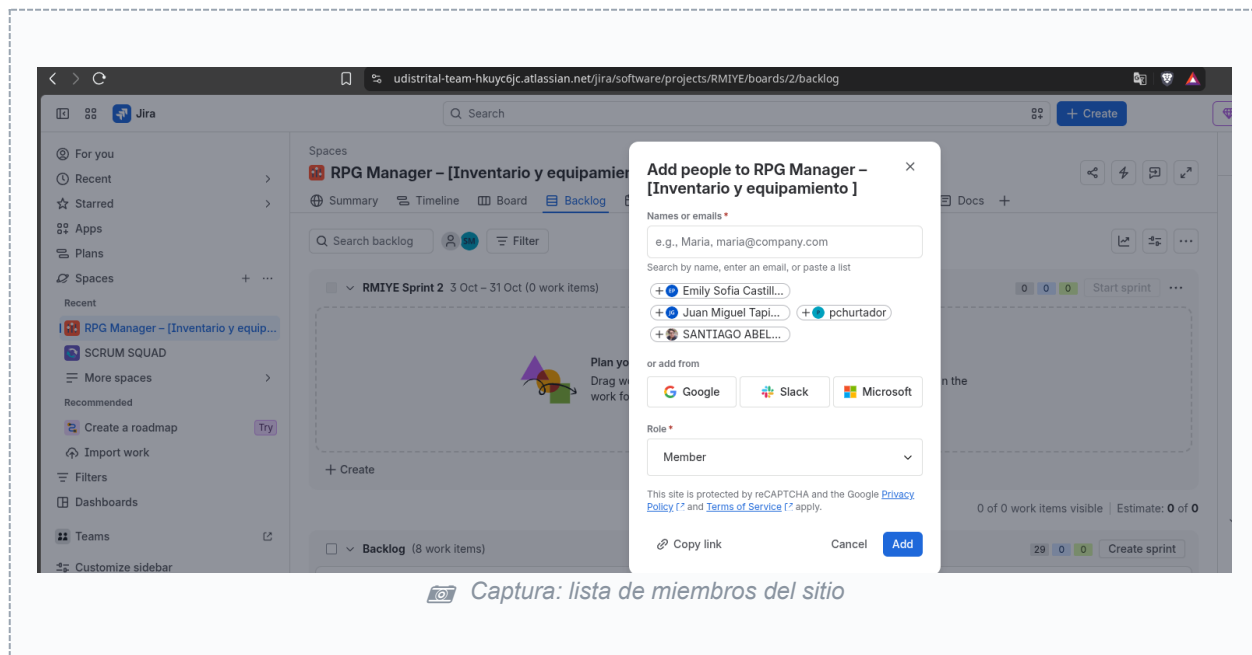
□ A1 · Crear el sitio y el proyecto

- Entra a atlassian.com y crea una cuenta gratuita con tu correo.
- Anota la URL del sitio: se verá como <https://<nombre>.atlassian.net>
- Create → Project → Software → plantilla Scrum.
- Nombre del proyecto: RPG Manager – [tu módulo].



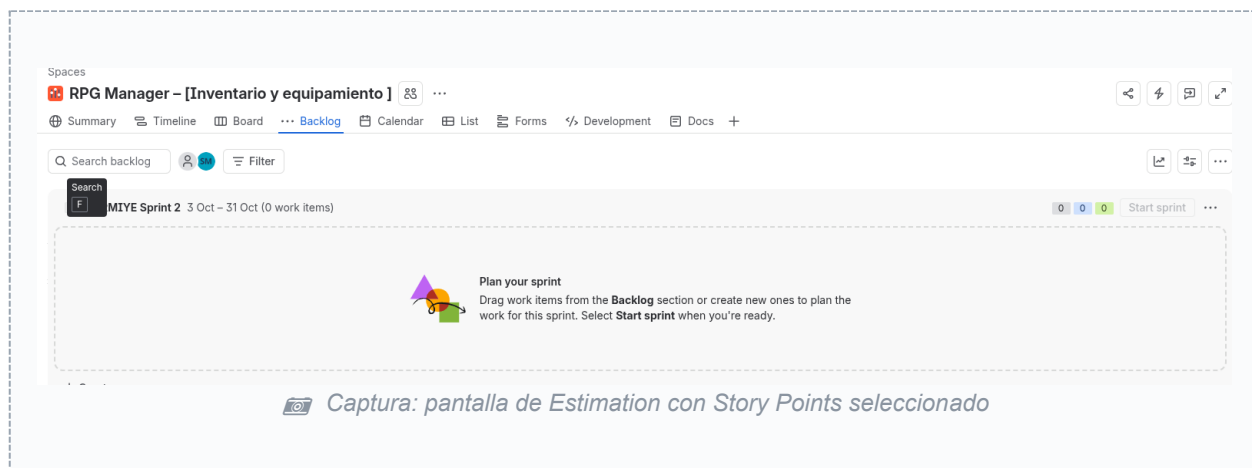
□ A2 · Invitar al equipo

- Menú de configuración (engranaje, arriba a la derecha) → People → Add people.
- Agrega el correo de cada integrante y el del profesor.
- El plan gratuito admite 10 usuarios: un equipo de 4 más el profesor cabe sin problema.



□ A3 · Activar la estimación en Story Points

- Abre la pestaña Backlog del proyecto.
- Menú "..." (arriba a la derecha del tablero) → Board settings → Estimation.
- Selecciona Story Points como unidad de estimación.
- Verifica que en el Backlog aparezca "Estimate: 0 of 0": ahí se acumularán los puntos del Sprint.



□ A4 · Preparar las etiquetas MoSCoW

- Abre cualquier historia y busca el campo Labels.
- Escribe Must y presiona Enter. Repite para Should, Could y Wont.
- Una vez creadas, las etiquetas aparecen como sugerencia en el resto de historias.
- No uses el campo Priority para MoSCoW: su escala significa otra cosa.

Add epic /

1

RMIYE-2

Como jugador, quiero ver la lista de objetos de mi inventario, para saber qué tengo disponible durante la aventura.

To Do

Agents

Improve Story

+

Description

CA1: Dado que recojo un objeto durante una aventura, cuando el objeto es obtenido correctamente, entonces aparece en mi inventario con su nombre, nivel y cantidad.

CA2: Dado que tengo objetos almacenados en mi inventario, cuando consulto el sistema muestra los objetos que poseo y a que nivel de armamento puedo aspirar.

Attachments

Add attachment

Subtasks

Add subtask

Start work

Details

Assignee

EP Emily Sofia Castillo Preciado

Assign to me

Parent

Add parent

Sprint

Add sprint +1

Labels

Must

Due date

Add due date

Team

Add team

Start date

Add date

Story point estimate

3

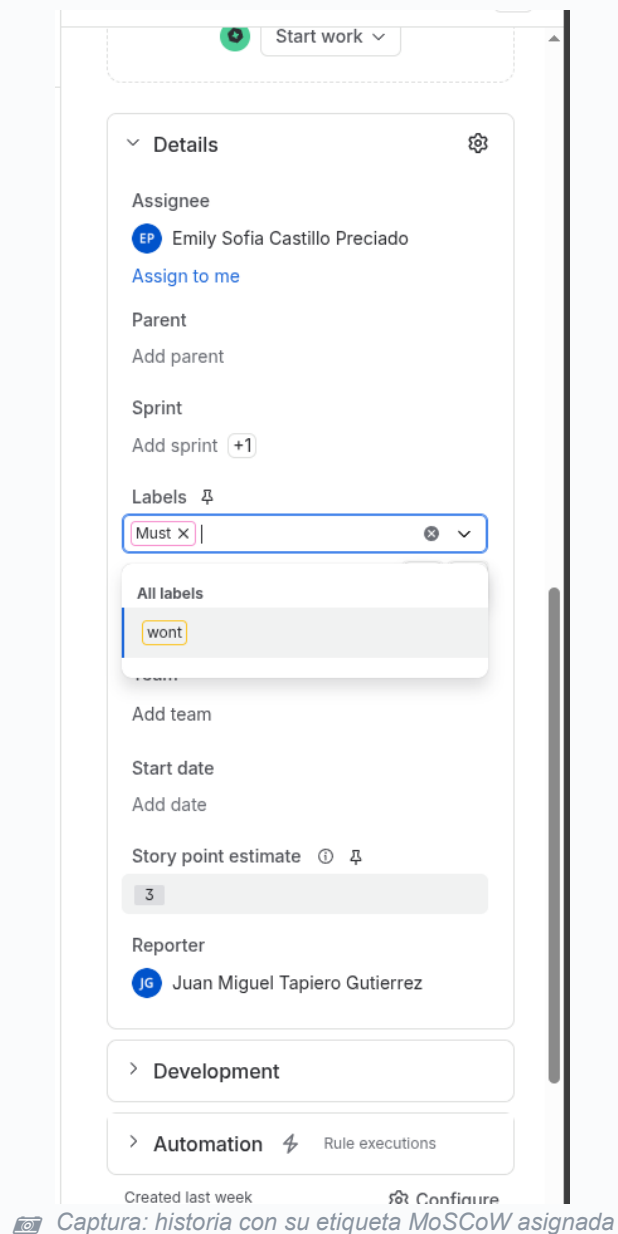
Reporter

JG Juan Miguel Tapiero Gutierrez

Development

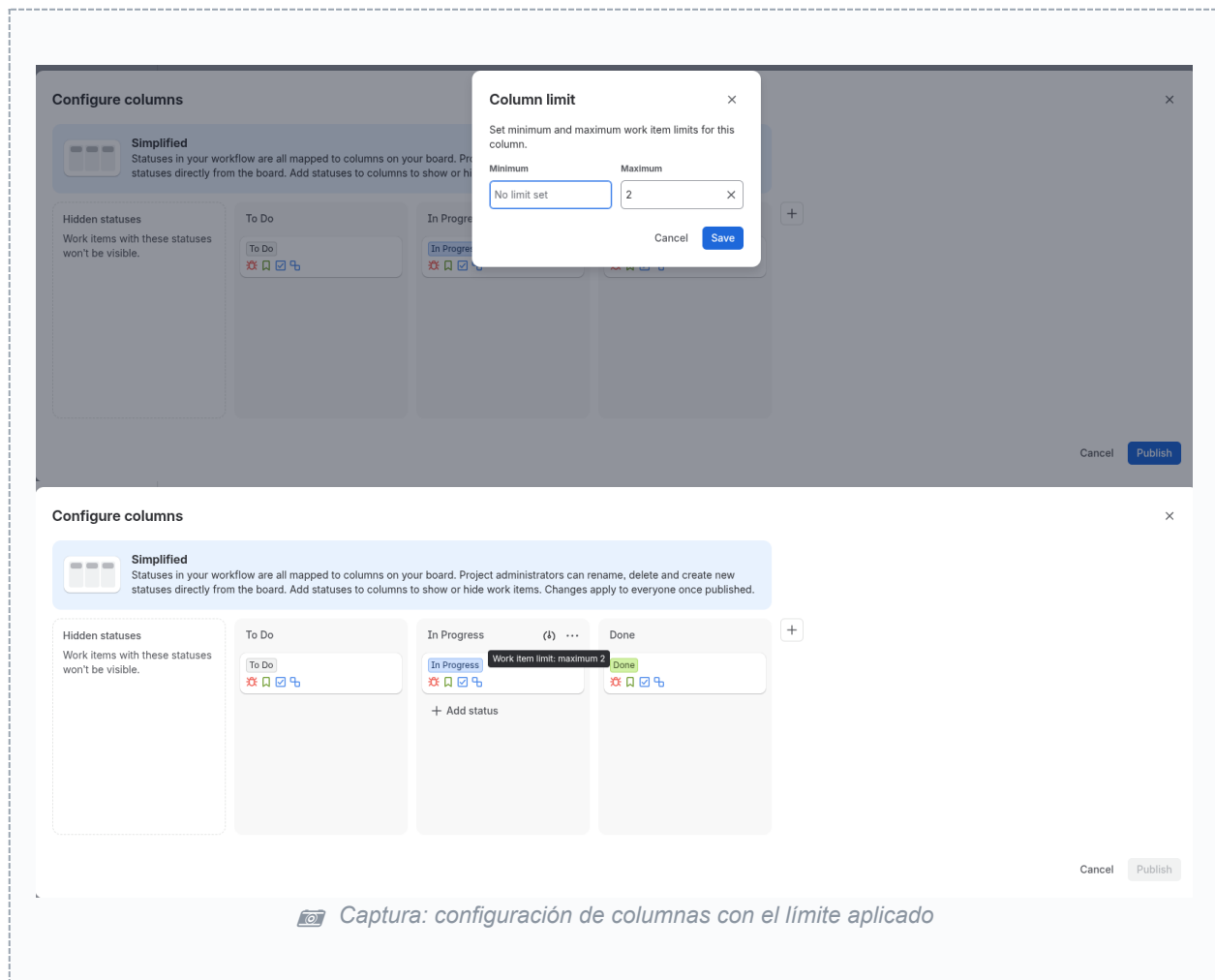
Automation

Rule executions



□ A5 · Configurar el límite WIP

- Board settings → Columns.
- En la columna In Progress, define Max issues = 2.
- Cuando el equipo supere el límite, la columna se resaltará en rojo en el tablero.



❑ A6 · Iniciar el Sprint

- En la pestaña Backlog, arrastra al Sprint las historias del Sprint Backlog (Bloque 5).
- Verifica que la suma de puntos no supere 13.
- Pulsa Start sprint. Asigna una duración corta y confirma.
- El trabajo pasa a la pestaña Board: ahí se mueven las tarjetas durante el Bloque 6.

RPG Manager – [Inventario y equipamiento]

SummaryTimelineBoardBacklogCalendarListFormsDevelopmentDocsReports

Search backlogFilter

RMIE Sprint 23 Oct – 31 Oct (3 work items)

1300Start sprint

RMIE-6Como jugador quiero tener un sistema de filtrado en el inventario, para que se me sea mas fácil buscar algoTo Do8

RMIE-3Como jugador, quiero equipar armas y armaduras de mi inventario, para mejorar las estadísticas de mi personaje.To Do3

RMIE-10Como jugador quiero tener un sistema de inventario, para conocer que objetos he recogido en la aventura.To Do2

Start Sprint

3 work items will be included in this sprint.

Required fields are marked with an asterisk *

Sprint name *

RMIE Sprint 2

Duration *

4 weeks

Start date *

9/9/202612:30 AM

Planned start date: 10/3/2026, 9:00 AM

A sprint's start date impacts velocity and scope in reports. [Learn more.](#)

Date format: MM/DD/YYYY. Time format: e.g. 1:00 PM.

End date *

10/7/202612:30 AM

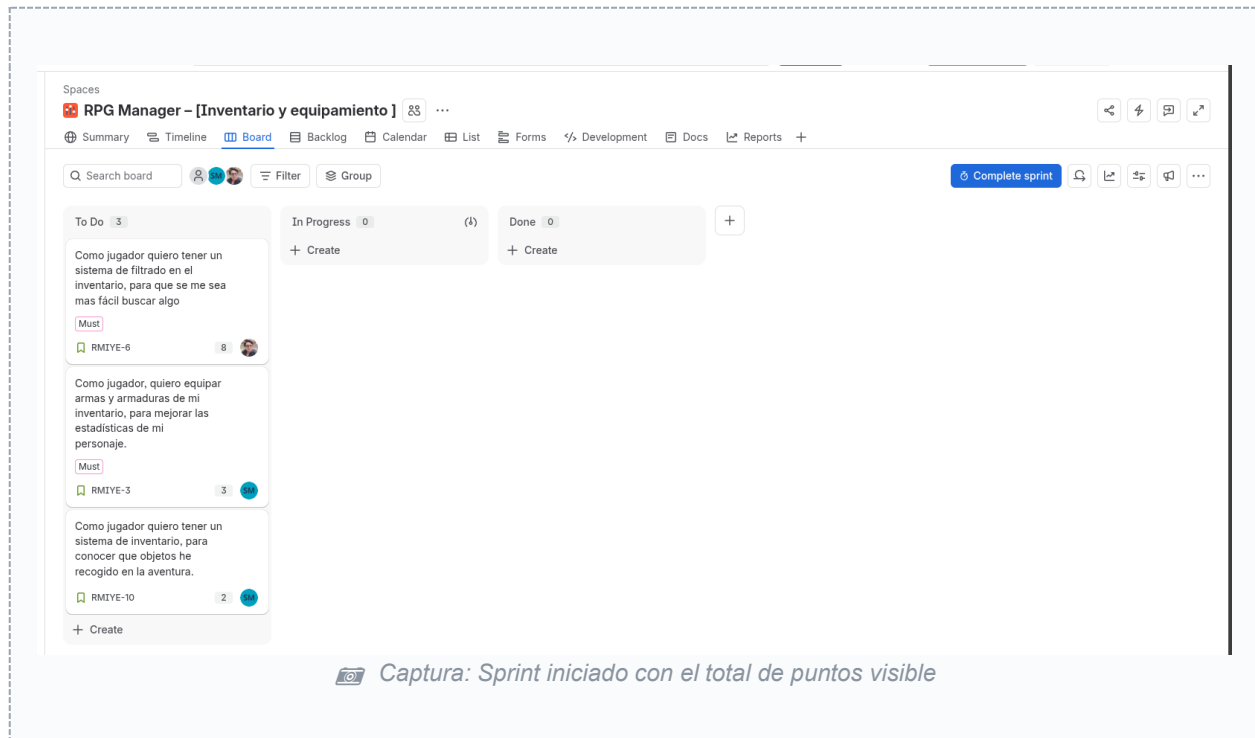
☒ Automatically complete sprint

When sprint completes, move open work to:

New sprint

Sprint goal

CancelStart



□ A7 · Cerrar el Sprint y ver el burndown

- Al terminar el Bloque 6, en la pestaña Board pulsa Complete sprint.
- Jira mostrará cuántas historias quedaron incompletas y preguntará a dónde moverlas.
- Ve al menú Reports → Burndown Chart para ver la gráfica de avance.
- Ambas pantallas son el material que presentarás en la Sprint Review.

Spaces
RPG Manager - [Inventario y e
Summary Timeline Board E
Reports +

Search board
Filter

To Do 1

Como jugador, quiero equipar armas y armaduras de mi inventario, para mejorar las estadísticas de mi personaje.

Must
RMIYE-3 3
+ Create

Complete RMIYE Sprint 2

This sprint contains **1 completed work item** and **2 open work items**.

- Completed work items includes everything in the last column on the board, **Done**.
- Open work items includes everything from any other column on the board. Move these to a new sprint or the backlog.

Move open work items to

New sprint

☒ **Join Confluence now to create a sprint retrospective**
Contribute to your team's culture and improve how you work. Let's get you set up quickly with a Confluence account.

Cancel
Complete sprint

Complete sprint

Spaces
RPG Manager - [Inventario y equipamiento]
Summary Timeline Board Backlog Calendar List Forms Development Docs Reports +

More reports

1 work item
completed in the last 7 days

7 work items
updated in the last 7 days

9 work items
created in the last 7 days

0 work items
due in the next 7 days

Work items by status

10
Total value

To Do Done

Work items by type

10
Total value

Story Epic

Work items by assignee

10
Total value

SAMUEL JULIAN C... SANTIAGO ABELLA... Emily Sofia Castillo ...

Work item creation trend

Done To Do

Work item cycle time



⚠️ NOTA SOBRE EL PLAN GRATUITO

El plan gratuito de Jira permite 10 usuarios y un solo sitio por cuenta. Por eso cada equipo crea su propio sitio en lugar de compartir uno solo: 6 equipos no caben en un único sitio gratuito.

Si tu correo ya tiene un sitio creado para otra materia, usa el de otro integrante del equipo.

No es un plan de prueba con fecha de vencimiento: el sitio del equipo sigue disponible después del semestre y puede servirte como evidencia en tu portafolio.

Preguntas de reflexión

🧠 RESPONDE INDIVIDUALMENTE

1. ¿En qué se parecen y en qué se diferencian un caso de uso UML y una historia de usuario? ¿Por qué un proyecto podría necesitar ambos?
2. ¿Por qué los story points miden esfuerzo relativo y no horas? ¿Qué problema tendría estimar directamente en horas?
3. Durante el Planning Poker, ¿qué pasó cuando dos personas dieron valores muy distintos? ¿Qué aprendió el equipo de esa conversación?

4. La capacidad del Sprint era fija en 13 puntos. ¿Qué habría pasado si el equipo hubiera decidido meter 18 puntos "para alcanzar a hacer más"?
5. ¿Qué relación tiene el límite WIP con la idea de que el trabajo a medias no entrega valor?
6. ¿Cómo se conecta el backlog que escribiste hoy con lo que se está programando en Programación Orientada a Objetos y arquitectando en Programación Multinivel?

RESPONDE INDIVIDUALMENTE-Samuel Julian Castillo Muñoz

1. ¿En qué se parecen y en qué se diferencian un caso de uso UML y una historia de usuario? ¿Por qué un proyecto podría necesitar ambos?

Se parecen en que los dos describen lo que un usuario necesita hacer en el sistema, pero su diferencia es que en caso de uso UML detalla con más precisión y de manera completa las acciones del usuario en el sistema, mientras que una historia de usuario describe la acción del usuario en una pieza pequeña que pueda caber dentro de un sprint. Un proyecto necesita ambos porque el caso de uso se usa en la fase de análisis, y la historia de usuario se escribe continuamente durante todo el desarrollo y pueda que de un caso de uso UML salgan muchas historias de usuario.

2. ¿Por qué los story points miden esfuerzo relativo y no horas? ¿Qué problema tendría estimar directamente en horas?

Los story points miden esfuerzo relativo porque comparan la complejidad y la cantidad de trabajo. Si se estimara en horas cada developer tardaría tiempos diferentes según la complejidad o experiencia.

3. Durante el Planning Poker, ¿qué pasó cuando dos personas dieron valores muy distintos? ¿Qué aprendió el equipo de esa conversación?

Cuando dos personas dieron valores distintos, cada uno tenía una perspectiva diferente de cómo dar solución a esa historia, al momento de cada developer exponer como lo harían, al final uno de los 2 llegaban a un acuerdo o se convencían entre si de su explicación y al momento de realizar nuevamente la calificación de la historia, coincidían en su puntaje.

4. La capacidad del Sprint era fija en 13 puntos. ¿Qué habría pasado si el equipo hubiera decidido meter 18 puntos "para alcanzar a hacer más"?

Si hubiéramos metido 18 puntos hubiéramos esforzado más la capacidad de los developers pero no para algo bueno ya que si hiciéramos así un sprint, probablemente quedaría bastante incompleto quedando sin desarrollar varias historias o hacerlas a medias al acabar el sprint, y eso es lo que hay que evitar

5. ¿Qué relación tiene el límite WIP con la idea de que el trabajo a medias no entrega valor?

El límite WIP de 2 tarjetas en este caso obliga al equipo a terminar una historia antes de empezar otra. Si no existiera ese límite, sería fácil que el equipo empezara muchas historias a la vez y terminara pocas, dando la falsa sensación de estar avanzando cuando en realidad no se completa nada.

6. ¿Cómo se conecta el backlog que escribiste hoy con lo que se está programando en Programación Orientada a Objetos y arquitectando en Programación Multinivel?

El backlog podría verse como programación orientada a objetos, con clases y atributos. Por ejemplo, una historia como equipar arma sería una clase Personaje con métodos para equiparla o usarla. En Programación Multinivel estaría la interfaz donde el usuario hace click en las armas y la lógica que valida la compatibilidad del arma, y la estructura de datos donde se guarda el inventario. El análisis que hacemos en historias de usuario define el características antes de que las otras materias definan como se construye técnicamente.

RESPUESTAS INDIVIDUALES – Paula Camila Hurtado Ramirez

1. ¿En qué se parecen y en qué se diferencian un caso de uso UML y una historia de usuario? ¿Por qué un proyecto podría necesitar ambos?

Ambos representan un registro de las necesidades del proyecto, la principal diferencia es su aplicación, el UML representando la lógica del proceso, mientras que la HDU se enfoca en lo que necesitan los usuarios finales.

2. ¿Por qué los story points miden esfuerzo relativo y no horas? ¿Qué problema tendría estimar directamente en horas?

Estimar las horas puede ser poco preciso y con una difícil trazabilidad, mientras que medirlo en story points se enfoca más en el nivel de dificultad de un problema y complejidad lo que ayuda a lograr tener una mejor comprensión en equipo del nivel de comparación entre un backlog a otro.

3. Durante el Planning Poker, ¿qué pasó cuando dos personas dieron valores muy distintos? ¿Qué aprendió el equipo de esa conversación?

La diferencia de conocimientos fue más evidente en algunas valoraciones, mientras unos lo miraban con un conocimiento previo de dificultades otros lo veían con lógica, pero no análisis de cuán difícil era completar ciertos requisitos.

4. La capacidad del Sprint era fija en 13 puntos. ¿Qué habría pasado si el equipo hubiera decidido meter 18 puntos "para alcanzar a hacer más"?

Probablemente el equipo habría terminado el Sprint con historias incompletas o con trabajo de menor calidad. Al superar la capacidad de 13 puntos, se estaría asumiendo un compromiso que el equipo no puede garantizar. Además, esto podría generar retrasos, presión y dificultades para cumplir los criterios de aceptación.

5. ¿Qué relación tiene el límite WIP con la idea de que el trabajo a medias no entrega valor?

Si todo está en proceso nada está terminado por eso, limitar el WIP ayuda a que el equipo se concentre en terminar las tareas antes de comenzar nuevas, aumentando la cantidad de trabajo realmente terminado y entregado.

6. ¿Cómo se conecta el backlog que escribiste hoy con lo que se está programando en Programación Orientada a Objetos y arquitectando en Programación Multinivel?

Los backlogs comunican las necesidades y requerimientos del código, donde se puede hacer la abstracción a clases y atributos en POO. También el análisis de conexiones entre capas de PM.

RESPUESTAS INDIVIDUALES – Emily Sofia Castillo Preciado

1. ¿En qué se parecen y en qué se diferencian un caso de uso UML y una historia de usuario? ¿Por qué un proyecto podría necesitar ambos?

Se parecen en que ambos capturan necesidades del usuario. Se diferencian en que el caso de uso detalla el cómo interactúa el actor con el sistema (flujos y precondiciones), mientras que la historia expresa brevemente el qué valor se busca entregar en un Sprint. Se necesitan ambos para tener una visión global en el diseño y una guía ejecutable para el desarrollo.

2. ¿Por qué los story points miden esfuerzo relativo y no horas? ¿Qué problema tendría estimar directamente en horas?

Miden esfuerzo relativo porque comparan la complejidad, el volumen de trabajo y la incertidumbre entre tareas. Estimar en horas es un problema porque varía según la experiencia de cada desarrollador y genera falsas expectativas de tiempo exacto ante imprevistos.

3. Durante el Planning Poker, ¿qué pasó cuando dos personas dieron valores muy distintos? ¿Qué aprendió el equipo de esa conversación?

Cada desarrollador explicó su razonamiento, visibilizando detalles técnicos que el otro no había considerado (como el filtrado por categorías). El equipo aprendió que discutir las diferencias ayuda a aclarar el alcance real antes de programar.

4. La capacidad del Sprint era fija en 13 puntos. ¿Qué habría pasado si el equipo hubiera decidido meter 18 puntos "para alcanzar a hacer más"?

El equipo habría sobrecargado su capacidad real, dejando historias a medias y entregando un Sprint incompleto y de menor calidad.

5. ¿Qué relación tiene el límite WIP con la idea de que el trabajo a medias no entrega valor?

El límite WIP obliga a terminar lo que está en progreso antes de iniciar algo nuevo. El trabajo al 80% no sirve al usuario; solo las historias terminadas en Done generan valor real.

6. ¿Cómo se conecta el backlog que escribiste hoy con lo que se está programando en Programación Orientada a Objetos y arquitectando en Programación Multinivel?

El backlog define las funcionalidades que en POO se traducen en clases y métodos (ej. `Inventario`, `equiparArma()`), y en Programación Multinivel guía la división entre capa de presentación, lógica de negocio y persistencia de datos.

RESPUESTAS INDIVIDUALES – Juan Miguel Tapiero Gutierrez

1. Como desarrollador, veo que ambos ayudan a entender qué debe hacer el sistema, pero tienen enfoques diferentes. La historia de usuario plantea la necesidad desde el punto de vista del usuario y qué valor espera obtener, mientras que el caso de uso UML permite detallar las interacciones y escenarios que tendría el sistema. Necesitamos ambos porque la historia me ayuda a entender qué funcionalidad debo desarrollar y el caso de uso me ayuda a entender mejor cómo se comportará esa funcionalidad.
2. Los story points permiten estimar una tarea comparándola con otras que ya conocemos, teniendo en cuenta su complejidad, esfuerzo y riesgo. Como desarrollador, considero que

estimar directamente en horas puede ser complicado porque pueden aparecer problemas técnicos que no estaban previstos y hacer que una tarea tome mucho más tiempo. Los puntos permiten hacer una estimación más flexible sin comprometer al equipo con un tiempo exacto.

3. Cuando hubo estimaciones diferentes, cada desarrollador explicó qué estaba teniendo en cuenta para asignar sus puntos. Eso permitió descubrir que no todos estábamos interpretando la historia de la misma manera o que algunos estábamos considerando dificultades técnicas que otros no habían visto. Como desarrollador, aprendí que la discusión de la estimación es tan importante como el número final, porque ayuda a detectar dudas y problemas antes de comenzar a programar.
4. Como desarrollador, creo que habría sido una mala decisión porque estaríamos comprometiendo más trabajo del que realmente podemos completar. Lo más probable es que algunas historias quedaran incompletas, aumentara el trabajo pendiente y posiblemente tuviéramos que continuar esas tareas en otro Sprint. Además, podríamos terminar priorizando terminar rápido sobre mantener la calidad del código.
5. El límite WIP ayuda a que como desarrolladores no tengamos demasiadas tareas abiertas simultáneamente. Tener cinco funcionalidades empezadas no significa que tengamos cinco funcionalidades entregadas. El valor aparece cuando una tarea está terminada y cumple sus criterios de aceptación. Por eso, el WIP nos obliga a enfocarnos en terminar lo que ya empezamos antes de seguir acumulando trabajo.
6. Como desarrollador, veo el backlog como el punto de partida de lo que después voy a implementar. Las historias de usuario indican las funcionalidades que necesita el sistema y los criterios de aceptación permiten saber cuándo una implementación está correctamente terminada. En Programación Orientada a Objetos, esas funcionalidades se convierten en clases, métodos y relaciones entre objetos; y en Programación Multinivel, se define cómo organizar esas funcionalidades entre las diferentes capas del sistema. Por eso, el backlog no está separado del código: lo que se define allí termina guiando las decisiones de implementación y arquitectura.

RESPUESTAS INDIVIDUALES – Santiago Abella Bello

1. Se parecen porque ambos describen lo que debe hacer el sistema. La diferencia es que el caso de uso es más detallado y la historia de usuario es más sencilla y enfocada en lo que necesita el usuario. Se pueden usar ambos para entender mejor qué se va a desarrollar.
2. Los story points miden qué tan difícil o qué tanto esfuerzo requiere una tarea, no un tiempo exacto. Si usáramos horas, sería más difícil calcularlas porque cada persona puede tardar diferente.
3. Cuando hubo valores diferentes, cada persona explicó por qué había elegido ese número. Eso nos ayudó a entender mejor la tarea y a darnos cuenta de que era importante hablar antes de decidir.
4. Si metíamos 18 puntos teniendo una capacidad de 13, probablemente quedarían tareas sin terminar o se harían con más errores. Lo mejor es respetar la capacidad del equipo.
5. El límite WIP evita tener muchas tareas empezadas al mismo tiempo. Así podemos enfocarnos en terminar una antes de comenzar demasiadas, porque una tarea a medias todavía no genera mucho valor.
6. El backlog nos dice qué funcionalidades necesita el sistema. Luego, en POO vemos cómo programarlas con clases y objetos, y en Programación Multinivel cómo organizarlas dentro de la arquitectura. En pocas palabras, el backlog dice qué hacer y las otras materias cómo hacerlo.

Autoevaluación del equipo

Marca con una X el nivel alcanzado en cada criterio.

Criterio	No logrado	En proceso	Logrado
Las 8 historias siguen la plantilla completa (Como... quiero... para...)	☐	☐	X
Cada historia tiene al menos dos criterios de aceptación verificables	☐	☐	X
La priorización MoSCoW respeta las cuotas y está justificada	☐	☐	X
Todas las historias fueron estimadas con Planning Poker en silencio	☐	☐	X
El Sprint Backlog respeta la capacidad de 13 puntos	☐	☐	X
Se respetó el límite WIP de 2 tarjetas en In Progress	☐	☐	X
Los incidentes fueron resueltos con una decisión argumentada	☐	☐	X
La retrospectiva produjo una acción concreta con responsable	☐	☐	X
El proyecto quedó configurado y visible en la herramienta	☐	☐	X
El commit fue realizado antes de salir del salón	☐	☐	X

CIERRE

Hoy tu equipo hizo lo que hace un analista en un proyecto ágil real: traducir necesidades en historias, ordenarlas bajo restricciones, estimar sin datos completos y sostener las decisiones frente a alguien que pregunta por qué.

La herramienta cambiará: Keisen, Jira, Azure DevOps o notas adhesivas. El razonamiento que practicaste hoy es el mismo en todas.